

Bloco 4.2, Propostas: types, helpers e layout-guard — status de entrega

Data: 2026-09-03. Branch: feature-dev-breno-automacao. Commit: 1de7ea5.

O que o bloco é

Preparar a base da entidade Propostas, sem tela: os tipos, os helpers puros e o guard de perfil que as telas dos blocos seguintes vão consumir. O padrão seguido é o de Orçamentos e FD —

`src/lib/types.ts` para os aliases de tabela, `src/lib/<entidade>.ts` para regra pura e `layout.tsx` com `requirePerfil`.

Atividade 1: conferir Proposta / PropostaInsert / PropostaUpdate

Concluída. Os três já existiam e estavam corretos.

`Proposta` já fazia o narrowing de `status` para `PropostaStatus`, porque o gerador do Supabase devolve `string` para coluna com CHECK constraint. Faltavam duas coisas, ambas feitas:

1. O comentário explicando o narrowing, que existe no `Orçamento` logo acima e não existia aqui.
2. O mesmo tratamento para `motivo_rejeicao`, que também é `text` com CHECK no banco e agora sai como `MotivoRejeicao | null`. A lista de motivos é a mesma de orçamentos, então reusa o `MotivoRejeicao` que já estava em `types.ts`.

`PropostaInsert` e `PropostaUpdate` continuam sendo os tipos crus do gerador, de propósito: quem escreve no banco passa pelos validadores da atividade 3.

Atividade 2: PropostaListItem com JOIN de obra e cliente

Concluída, com uma correção de premissa.

A tarefa pede "JOIN de obra (codigo_obra, nome) e cliente", o que sugere dois JOINS irmãos, como em `OrçamentoListItem`. Não é possível: **propostas não tem `cliente_id`**. A tabela liga em obra pela FK composta `propostas_obra_fk` (`obra_id`, `empresa_id`), e o cliente vem por obra. O JOIN é aninhado, `propostas → obras → clientes`:

```
obra: (Pick<Obra, 'codigo_obra' | 'nome'> & {
  cliente: Pick<Cliente, 'nome'> | null
}) | null
```

O select correspondente, para as telas do próximo bloco:

```
id, numero, data_emissao, data_validade, obra_id, status,
valor_total, desconto, valor_final,
obra:obras(codigo_obra, nome, cliente:clientes(nome))
```

`valor_total`, `desconto` e `valor_final` entram juntos porque `valor_final` é coluna **generated** (`valor_total - desconto`), e a listagem precisa mostrar o desconto sem uma segunda consulta.

Atividade 3: src/lib/propostas.ts

Concluída. 275 linhas, sem `use client` e sem import de React, então serve Server Component, Server Action e Client Component igual ao `orcamento-form-helpers.ts`.

Status

EXPORT	O QUE FAZ
<code>STATUS_PROPOSTA_LABELS</code> , <code>formatStatusPropostaLabel</code>	rótulos em português, mesmo formato do <code>fd.ts</code>
<code>STATUS_PROPOSTA_OPTIONS</code>	lista pronta para <code><select></code> e para o diálogo de status
<code>isPropostaStatus</code>	type guard para <code>querystring (?status=)</code> e input externo
<code>requiresDataEnvio</code> , <code>requiresDataDecisao</code> , <code>requiresMotivoRejeicao</code>	quais campos cada status exige
<code>isEditavel</code> , <code>isFinalizada</code>	rascunho é o único editável; aprovada e rejeitada são terminais
<code>PROPOSTA_TRANSICOES</code> , <code>canChangePropostaStatus</code>	fluxo permitido entre status
<code>isPropostaVencida</code>	validade expirada, derivada em runtime
<code>validarMotivoRejeicao</code>	espelha o CHECK <code>propostas_rejeitada_motivo</code>

Duas decisões que valem registro, porque não vêm do banco:

O fluxo de status é regra de aplicação. O CHECK do banco valida só a lista de valores (`rascunho`, `enviada`, `aprovada`, `rejeitada`), não a ordem. O que ficou codificado é `rascunho → enviada`, `enviada → aprovada | rejeitada | rascunho` (voltar para correção) e `aprovada/rejeitada` terminais. Se o fluxo comercial real permitir reabrir uma decisão, é uma linha em `PROPOSTA_TRANSICOES`.

Proposta vencida não é status. Diferente de orçamento, que tem `expirado` no banco, aqui não existe `expirada`. `isPropostaVencida` compara `data_validade` com hoje e só vale para proposta `enviada` e ainda sem decisão — mesmo padrão do `computeStatusFd` em `fd.ts`, que também deriva em runtime.

Percentuais de pagamento

O ponto que mais exigia cuidado. No banco os quatro `pct_*` são `numeric(5,4)` com `CHECK between 0 and 1`, ou seja **frações**, e existe o `CHECK propostas_pct_soma` garantindo que a soma dos `coalesce(pct_*, 0)` não passe de `1.0`. A UI trabalha em 0 a 100.

EXPORT	O QUE FAZ
<code>PCT_FIELDS</code> , <code>PctField</code> , <code>PCT_LABELS</code> , <code>PctValues</code>	os quatro campos, rótulos e o shape parcial
<code>somaPct</code> , <code>pctRestante</code> , <code>isPctCompleto</code>	soma com <code>coalesce</code> , o que sobra e se fecha 100%
<code>validarSomaPct</code>	espelha os dois CHECKs e devolve <code>soma</code> e <code>restante</code>
<code>pctToFraction</code> , <code>fractionToPct</code> , <code>formatPct</code>	conversão nas bordas e "12,5%" em pt-BR
<code>valorBaseProposta</code> , <code>valorParcela</code> , <code>parcelasDaProposta</code>	rateio em reais sobre o valor final

Três detalhes deliberados:

- Arredondamento em 4 casas antes de comparar.** `numeric(5,4)` só guarda quatro decimais, e `0.1 + 0.2` em float dá `0.30000000000000004`. Sem o `round4`, uma soma legítima de 100% seria recusada pelo validador na frente de um banco que a aceitaria.
- Soma abaixo de 100% é válida.** O CHECK é `<= 1.0`, não `= 1.0`. O resto fica para o campo livre `condicoes_pagamento`, então `validarSomaPct` devolve `restante` em vez de barrar, e `isPctCompleto` existe para a tela sinalizar sem bloquear o salvamento.
- A base de rateio é `valor_final`.** É generated no banco; `valorBaseProposta` tem fallback para `valor_total - desconto` só para o caso do registro em memória que ainda não voltou do insert.

Atividade 4: layout.tsx com guard de perfil

Concluída. Já estava correto, nada a fazer.

`src/app/(app)/propostas/layout.tsx` já chamava `requirePerfil(['admin', 'comercial', 'visualizador'])`, idêntico ao de orçamentos, e o item de menu em `src/lib/nav.ts` já usa a mesma lista de perfis. Conferido que os dois batem, para não existir link visível que redireciona.

Feito além do checklist

StatusBadge ganhou as **chaves de proposta**. O componente conhecia só os status masculinos de orçamento (`aprovado`, `rejeitado`) e os de obra. Os de proposta são femininos, chaves diferentes, então caíam no fallback cinza com o label cru. Foram adicionadas as quatro (`rascunho`, `enviada`, `aprovada`, `rejeitada`) com as mesmas cores das equivalentes de orçamento, e `StatusKey` passou a incluir `PropostaStatus`.

Duas implementações paralelas foram consolidadas em uma. Existia o commit `24cbc51`, na branch auxiliar `worktree-bloco-4-2-propostas-base`, e um segundo trabalho não commitado no checkout principal, cobrindo os mesmos três arquivos. Nada foi descartado: a versão da branch entrou como base, por já estar commitada e por trazer as conversões de borda (`pctToFraction` / `fractionToPct`), o `round4`, o `formatPct` em pt-BR e o narrowing de `motivo_rejeicao`; a versão local somou `PROPOSTA_TRANSICOES`, `canChangePropostaStatus`, `isPropostaVencida`, `validarMotivoRejeicao`, `valorBaseProposta`, `pctRestante` e `isPctCompleto`. Sem nome duplicado: `validarMotivoRejeicao` reusa `requiresMotivoRejeicao`, e `parcelasDaProposta` passou a usar `pctRestante`.

As branches auxiliares foram apagadas, conforme a regra de trabalhar só na branch de trabalho. Saíram `worktree-bloco-4-2-propostas-base` e `worktree-verificar-schema-4-1` (essa não tinha commit exclusivo, já era ancestral da branch de trabalho), com os dois worktrees removidos e o `git worktree prune` rodado. Restam `feature-dev-breno-automacao` e `main`.

Verificação

COMANDO	RESULTADO
<code>npx tsc --noEmit</code>	limpo
<code>npx eslint</code> nos três arquivos	limpo

Não há teste automatizado no repositório, então os helpers de `pct_*` e de status não têm cobertura. Eles são funções puras e sem dependência, o que os torna o primeiro candidato natural quando entrar um runner (ver pendências).

O que vem depois

O bloco 4.2 entrega base, não tela. `src/app/(app)/propostas/page.tsx` continua em `<EmConstrucao />`. Seguindo o que Orçamentos já tem, é isto que falta para Propostas ficar completa:

- Listagem.** `page.tsx` com o select da atividade 2, mais `filters.tsx` e `propostas-table.tsx`. Lição do 4.1 que se aplica aqui: usar `as` e nunca `as unknown as` no retorno do `select()`, senão o compilador para de comparar a string da query com o `Database` gerado e o `tsc` deixa de acusar coluna inexistente.
- Criação e edição.** `proposta-form-helpers.ts` com schema zod e o par `formValuesToPayload` / `propostaToFormValues`, mais `proposta-form.tsx`. É onde `pctToFraction` e `fractionToPct` entram: o form em 0 a 100, o payload em fração. O schema precisa reproduzir `desconto <= valor_total` (CHECK `propostas_desconto_valido`) e chamar `validarSomaPct` antes de submeter. A seleção de obra substitui a de cliente de orçamentos, e o `numero` é obrigatório em proposta, diferente de orçamento.
- Detalhe e mudança de status.** `[id]/page.tsx`, `detail-header.tsx` e `status-dialog.tsx` sobre `canChangePropostaStatus`, `requiresDataEnvio`, `requiresDataDecisao` e `validarMotivoRejeicao`. O diálogo precisa limpar `motivo_rejeicao` e `detalhe_rejeicao` quando o status sai de `rejeitada`,

porque o CHECK do banco proíbe motivo em status diferente de rejeitada, igual ao que `changeOrcamentoStatus` já faz.

4. **Anexos.** `anexos-tab.tsx` sobre a coluna `anexos` jsonb e o bucket `anexos`, reusando `buildStoragePath`, `validateFile` e o tipo `Anexo`.
5. **Permissão de escrita nas Server Actions.** O guard de layout protege a rota, não a ação. As actions precisam repetir o padrão de orçamentos: admin e comercial mudam status e editam, só admin exclui. E vale lembrar que `propostas_obra_fk` é `ON DELETE RESTRICT`, então obra com proposta não é deletável — o erro do banco tem que virar mensagem legível.
6. **Exportação.** `api/export/propostas`, no formato dos três que já existem.
7. **Painel financeiro da proposta**, se entrar no escopo: a view `propostas_financeiro` já existe e traz `total_nfs`, `recebido_nfs`, `total_acordos` e `recebido_acordos`, e o alias `PropostaFinanceiro` já está em `types.ts`.

Pendências

1. **Sem cobertura de teste.** O repositório não tem runner. `validarSomaPct`, `pctToFraction` e `canChangePropostaStatus` são puros e mereciam teste antes das telas dependerem deles.
2. **Confirmar o fluxo de status** de `PROPOSTA_TRANSICOES` com a área comercial, em especial se proposta aprovada ou rejeitada pode voltar ao funil.
3. **Definir o tratamento de proposta vencida na UI.** `isPropostaVencida` está pronto, mas falta decidir se aparece como selo ao lado do status, como filtro na listagem, ou os dois.
4. Herdadas do 4.1 e ainda abertas: `supabase migration repair --status applied` nas dez versões antes de qualquer `db push`, revogar o personal access token usado para gerar os types, e `gc-prod` quando despausar.

Estado do repositório

Tudo commitado em **feature-dev-breno-automacao**, que foi de `0f16abf` para `1de7ea5`. Nenhum worktree ativo, só as branches `feature-dev-breno-automacao` e `main`. O push para o GitHub continua bloqueado: `brenocml0` não tem permissão de escrita em `cremalet/gc-sistema`, então os commits existem só no clone local.

Duas coisas ficaram fora deste commit, de propósito, porque não são do 4.2 e apagar seria destrutivo: `docs/4.1-status-entrega.{html,md,pdf}` e `package-lock.json` aparecem como deletados no working tree sem estar em stage, de antes deste trabalho, e o diretório `.claude/` segue untracked.

Arquivos

ARQUIVO	O QUE É
<code>src/lib/propostas.ts</code>	novo. Helpers de status e de <code>pct_*</code>

ARQUIVO	O QUE É
<code>src/lib/types.ts</code>	<code>PropostaListItem</code> novo, narrowing de <code>motivo_rejeicao</code>
<code>src/components/StatusBadge.tsx</code>	as quatro chaves de status de proposta
<code>src/app/(app)/propostas/layout.tsx</code>	inalterado, já tinha o guard correto
<code>docs/4.2-status-entrega.md</code>	este documento